

Salesforce Migration Guide

Airflow to Databricks Jobs Orchestrator

Generated from existing migration documentation

Airflow to Databricks Salesforce Migration Guide

This guide shows how to migrate a typical Airflow Salesforce DAG to the Databricks Python Operator task pattern implemented in this repository.

Why migrate

- ? Reduce orchestration glue code (custom Airflow operators/sensors).
- ? Keep task runtime, retries, logs, and deployment in one Databricks surface.
- ? Reuse the same Salesforce primitives as either functions or operators.

Concept mapping

```
Airflow pattern | Databricks equivalent in this repo
--- | ---
'PythonOperator' calling Salesforce SDK | 'salesforce_api.salesforce_insert' /
'salesforce_upsert' / 'salesforce_update'
Custom 'BaseOperator' for long-running Salesforce bulk jobs |
'salesforce_api.SalesforceBulkWriteOperator' / 'SalesforceUpsertOperator'
Separate 'Sensor' task waiting on bulk job completion | Operator 'poll()' lifecycle or
function with 'wait_for_completion=true'
XCom handoff of job IDs and counters | 'dbutils.jobs.taskValues'
Airflow DAG deployment + env packaging | Databricks Asset Bundle ('databricks.yml' +
'resources/*.yml')
```

Before: actual Airflow DAG in this repo

Source DAG file:

? 'airflow_dags/salesforce_bulk_migration_source_dag.py'

That DAG is the concrete migration source artifact for this hackathon.

After: Databricks migrated workflow

Use the job defined in:

? 'resources/salesforce_airflow_migration.yml'

It includes:

1. 'airflow_migrated_insert_fn' (function)
2. 'airflow_migrated_upsert_operator' (operator)

3. 'airflow_migrated_update_fn' (function)

This demonstrates a practical migration path where you keep straightforward steps as functions and move lifecycle-heavy steps to operators.

Migration checklist

1. Move connection details into UC HTTP connection ('salesforce_m2m_conn').
2. Replace custom Salesforce SDK calls with repo primitives:
? 'salesforce_insert', 'salesforce_upsert', 'salesforce_update'
? 'SalesforceBulkWriteOperator', 'SalesforceUpsertOperator'
3. Translate task dependency graph from Airflow to Databricks job tasks.
4. Replace XCom payloads with 'taskValues' only when needed.
5. Deploy and test with:
? 'databricks bundle deploy --profile dogfood'
? 'databricks bundle run salesforce_airflow_migration_demo --profile dogfood'

Notes

- ? Functions are blocking when 'wait_for_completion=true'.
- ? Operators encapsulate 'open/poll/close' for external job lifecycle control.
- ? For hackathon demonstrations, this migration pattern is the direct answer to "How do we replace Airflow operator maintenance with Databricks-native tasks?"